Microprocessor.

Microcontroller

- It is a specialized form of microprocessor designed for specific embedded system applications. It contains CPU, RAM, ROM, Registers, Timer and I/O ports.
- It is a self-sufficient unit, a chip called single chip computer.
- Lower processing power
- Fixed instruction set.
- Lower clock speed (~100MHz)
- Low power consumption.
- Low cost

ex. TV

Microprocessor

- It is designed for general purpose computing with a combination of timers, controllers and memory chips.
- It is a dependent unit.
- Higher processing power.
- More flexible instruction set.
- Higher clock speed (~1GHz)
- High power consumption
- Highe cost

ex. Personal computer.

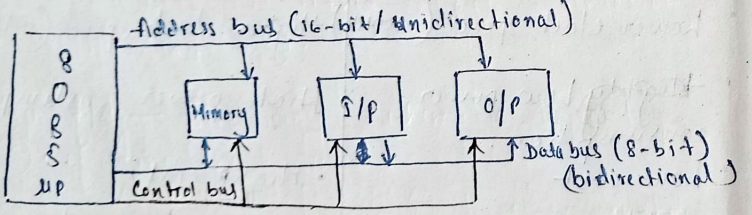
- 4 bit μp – 4040 series
- 8 bit μp – 8080 series

[8085 – enchanced with execution time in μs]

- 16 bit μp – 8086 μp, 8088, ...

Microprocessor — Microprocessor is a multipurpose programmable clock driven register based integrated circuit that can read instruction from memory, take data from input and process it according to the instruction and provide result as the output.

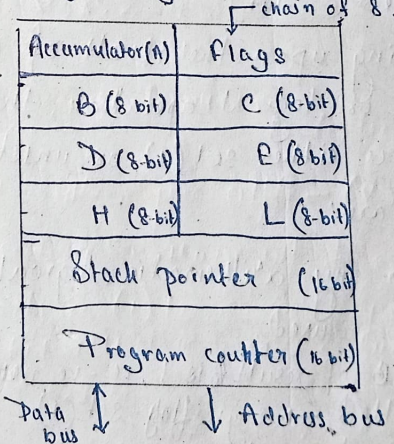
8085 Bus Structure.



Features of 8085 μp.

- 8085 μp is a 8-bit μp since ALU structure is 8-bit.
- It has a 8-bit wide data bus
- It can access 64 KB = 2^{16} bytes of memory [16 bit address bus]
- The least significant 8-bit address bus and the data bus are multiplexed.
- The device has 40 pins and require single 5 V power supply
- It can operate with a 3-MHz clock.

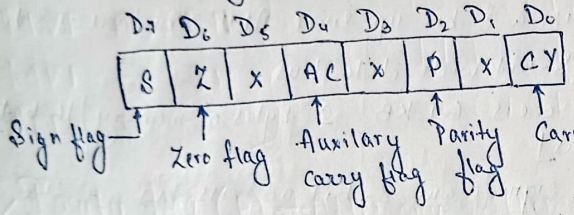
Programming / Software mode of 8085 μp



Register pair

- B → ⟨B, C⟩ 16-bit
- D → ⟨D, E⟩ 16-bit
- H → ⟨H, L⟩ 16-bit

Flag structure of 8085 µp —



Flag register — The flag register is a special purpose register - consisting of 8 flip flops In 8085 microprocessor, the flag register has 8 flip flops (8-bits) but only 5 of them are used (5-bit).

Dependending upon the result of any arithematic or logical operation, the flag bits are either set (1) or unset (0) The flags are:

1) Sign flag (S) — After any arithematic, or logical operation
- if signed no is used,
  - if MSB of result is 1, ie -ve number is resulted → sign flag - set (1)
  - if MSB of result is 0, it +ve number result → sign flag - reset (0)
- if unsigned no is used, sign bit is ignored.

2) Zero flag (Z) — After any operation, if ~~accumulator content is~~ result is zero, then zero flag is set (1) otherwise unset

Examples:

- Sign flag.

```
MVI A, 40H
MVI B, 50H
SUB B
```

sets the sign flag as the result will be negative, stored in A.

- Zero flag.

```
MVI A, 10H
SUB A
```

sets zero flag as result will be 0, stored in A.

3) Auxilary carry flag (AC) — This flag is used in the BCD number system.
It is set (1), if the lower 4-bit generates a carry (D_3 generates carry or borrows)
It is reset (0), otherwise.

* It is the only flag not accessible by the programmer.

Example

```
MVI A, 2BH
MVI B, 39H
ADD B
```

CY from D_3 to D_4 → AC

```
  00101011   <set AC flag
  00111001    to 1>
-----------
  0100
```

4) Parity flag (P) – After any arithmetic or logical operation if the accumulator content has even no of 1 bits then parity flag is set (1) or else reset (0).

Example,

```
MVI A, 05H .   05H → <AC>
```

00000101

even no of 1s.

Parity flag is set (1)

5) Carry flag (CY) – After any arithmetic or logical opn. if the result is greater than 8-bit, carry flag is set (1) or else reset (0) - it carry-out or borrow in-in MSB bit.

Example,

```
MVI A, 30H
MVI B, 40H
SUB B
```

Borrow-in in MSB bit

* Sets carry flag

Use of flag register:

It contains status of the arithmetic logic opn, and is used by processor to make decisions based on the status.

1) Conditional branching – flags are used to determine which instruction to branch to. ex. Jump to Zero (JZ) checks zero flag.

2) Arithmetic operation – flags are used in almost every arithmetic opn. – addition, sub, mul, div. ex. CY and AC.

3) Logical op – flags used during logical opn. – AND, OR, XOR.

4) System calls – flags are used by OS to determine the status of CPU before and after a system call.

Adv: i) simplifies branching
ii) faster execution
iii) saves memory
iv) efficient error detection.

Registers in 8085 μp:

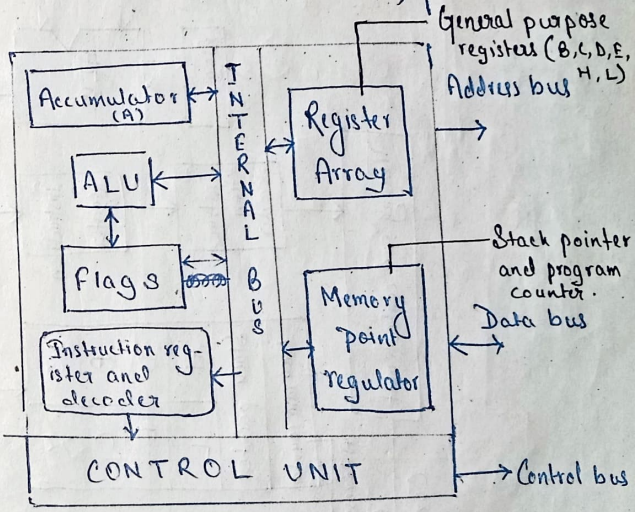
1) General purpose register - 6 general purpose registers to store 8-bit data - B, C, D, E, H, L.
   Forms register pair <BC>, <DE>, <HL>
   Use - temporarily store data during computation.
   - used in pair (16-bit) to hold addresses for efficient addressing of memory location.

2) Special Specific purpose register -
   • Accumulator - 8-bit data register used to store the begin, intermediate and result of ALU opn. Source of destination of every ALU opn. and also used for I/O opn.
   • Flag registers.

3) Memory register.
   • Program counter - Program counter is essential to maintain the sequence of instruction execution. It holds the address of instruction executed immediately.

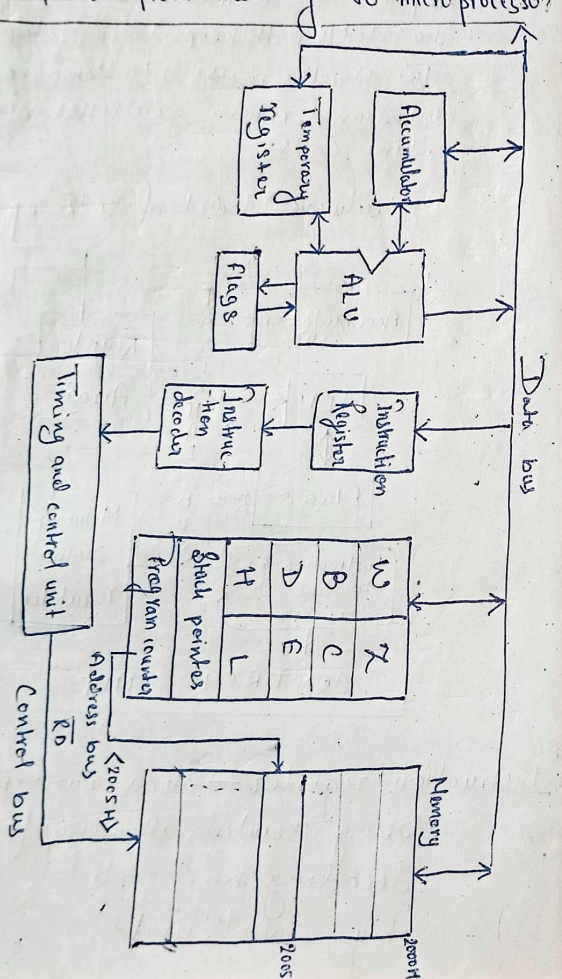
• Stack pointer - Used as a memory pointer to point to a memory location in read/write memory called the stack. It keeps track of top of stack. The stack is used to temporarily store data and return addresses during sub-routine calls.

Hardware Model of 8085 μp

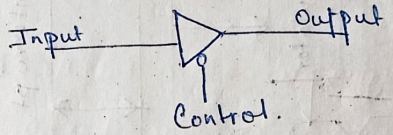


• Instruction register - Stores current instruction being executed, allowing for efficient decoding and control signal generation by control unit.

Data-flow from memory to microprocessor

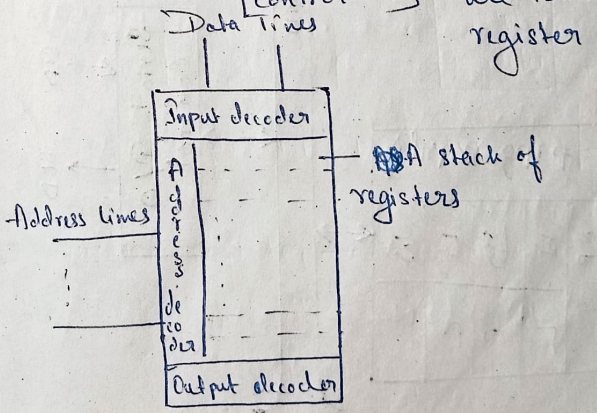


Data / Control / Address bus are connected to memory through a buffer called tri-state buffer.



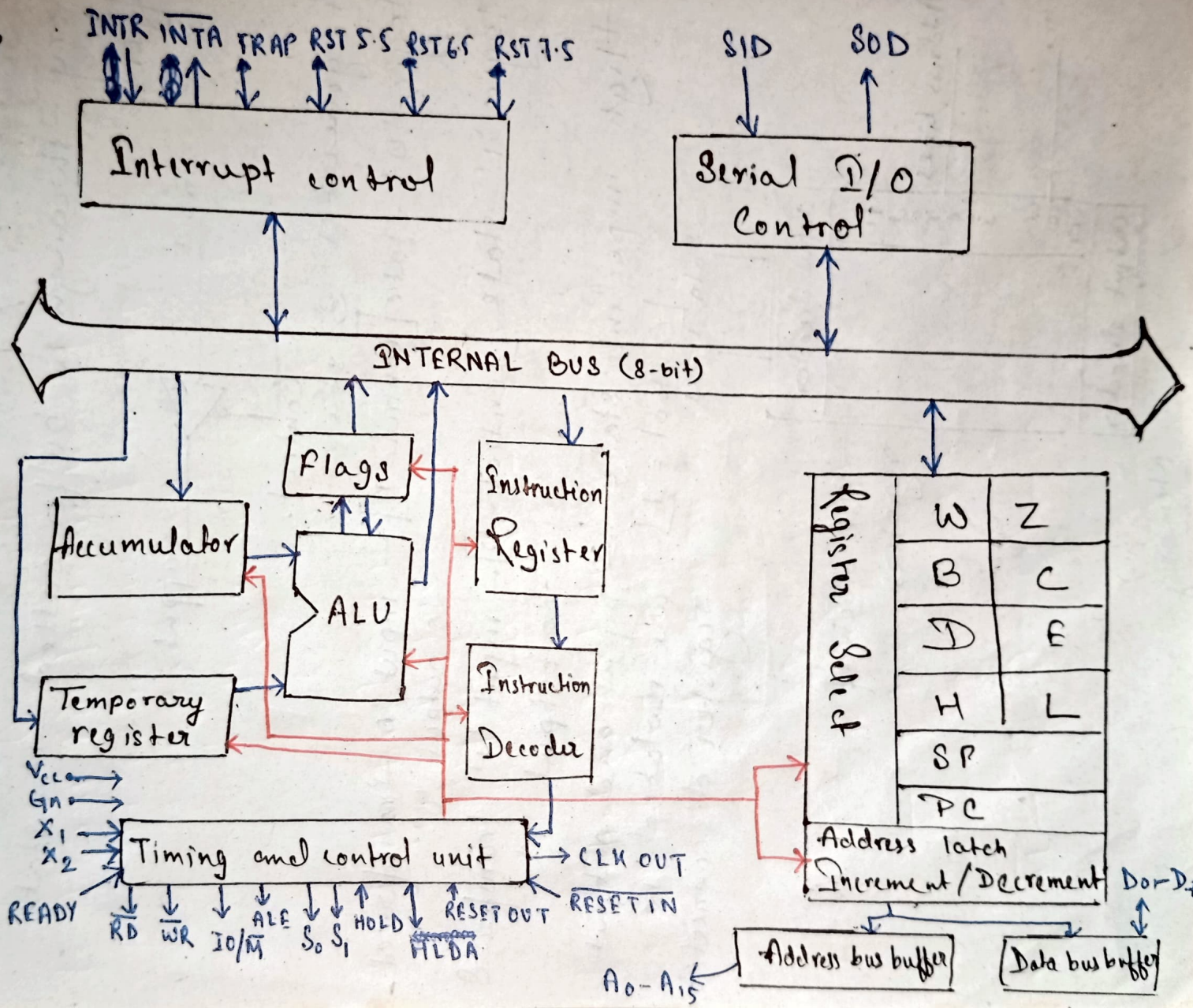
It generates 3 states:

1) logic '0' state [Control = 0] – input '0' transferred to output '0'
2) logic '1' state [Control = 0] – input '1' transferred to output '1'
3) High impedence state [Control = 1] – input and output are isolated; ie register disconnected.



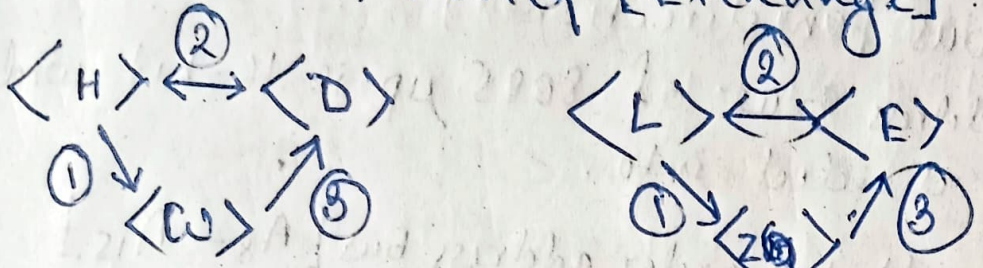
Memory

Architecture of 8085 µP / Function Block diagram of 8085 µP

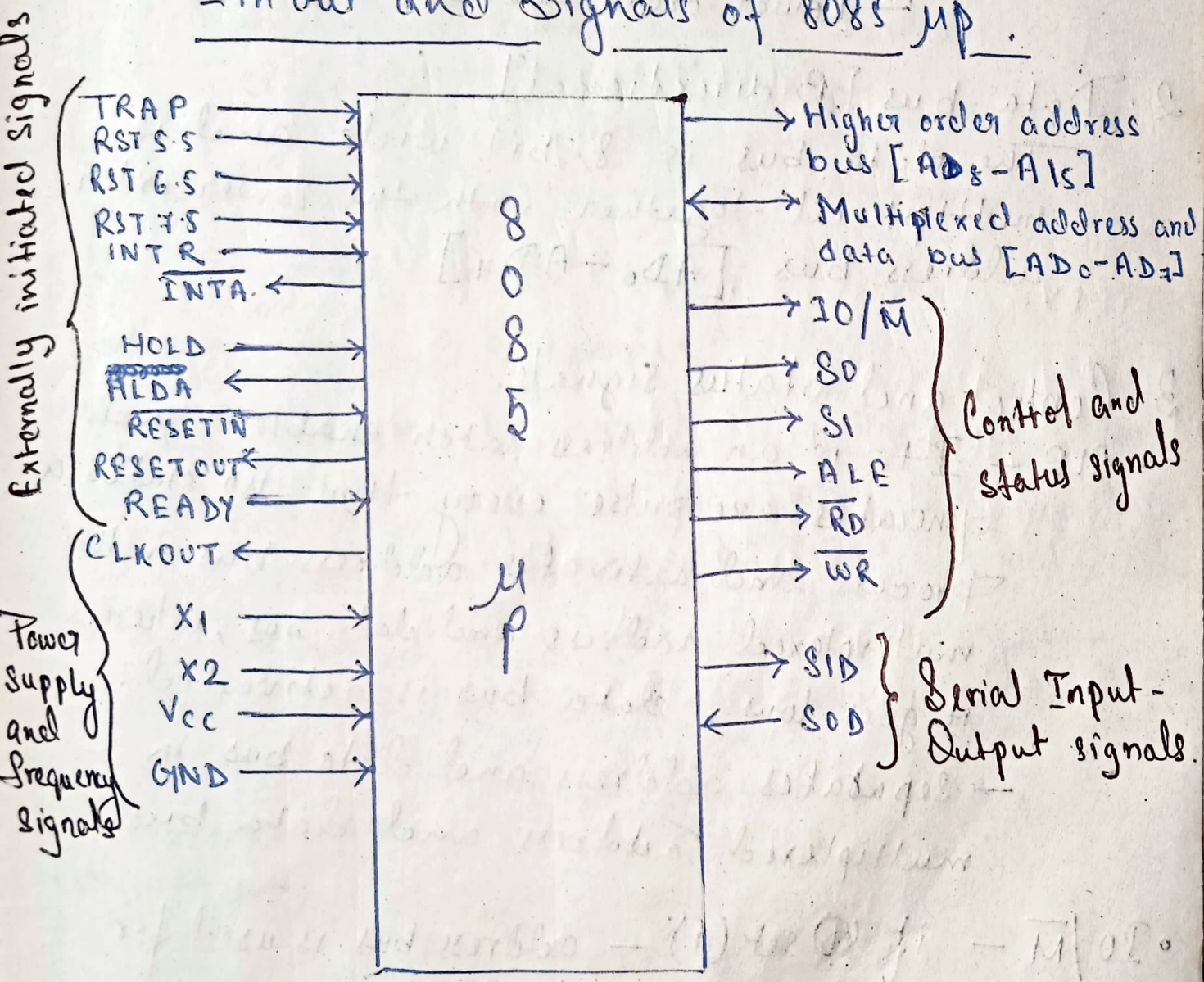


W, Z are temporary registers.

On execution of XCHG [Exchange]



Pin out and Signals of 8085 µp.



Signals of 8085 µp:

1. Address bus [unidirectional]

The address bus of 8085 µp is 16-bit wide. It is divided into.

- Higher order address bus [$A_8 - A_{15}$]
- Lower order address bus [$A_0 - A_7$]

2. Data bus [Bidirectional]

The data bus is 8-bit wide and is multiplexed together with the lower order address bus [$AD_0 - AD_7$]

3. Control and Status signals.

- ALE – It is an address latch enable signal. Generates +ve pulse every time µp starts a process and activates address bus in the multiplexed address and data bus, when it goes low, data bus is activated.
  – Separates address and data bus in multiplexed address and data bus.
- IO/$\overline{M}$ – if set (1) – address bus is used for I/O devices.
  if reset (0) – address bus is used for memory
- S0, S1 – Status signals that help to different types of instructions such as halt, reading, writing or inst. fetching.
- $\overline{RD}$ – Controls READ operation. When 0, the selected I/O or memory is read.
- $\overline{WR}$ – Controls WRITE operation. When 0, the data on data bus is written in the selected I/O or memory.

4. Power supply and frequency signal

- V_{cc} – +5V power supply
- GND – Ground reference.
- X1, X2 – Crystal is connected to the pins whose frequency is internally divided by two. If $f_c = 3$ MHz, we need $f_c = 6$ MHz
- CLK OUT – Signal can be used as system clock for other devices.

5. Externally initiated signals.

- Interrupts (Emergency pins) – 8085 has 5 interrupts to cease a program execution
  1) INTR 2) RST 5.5 3) RST 6.5 4) RST 7.5 5) TRAP.

* INTR – It is a non-vector interrupt request signal.

* $\overline{INTA}$ – It is an interrupt acknowledgement sent by µp after INTR is received.

- **Reset signals**

  RESET IN – when '0', PC = 0, buses are tristated and µp unit is reset.

  RESET OUT – signal indicated µp unit is being reset. can be used to reset other devices.

- READY – It senses whether a peripheral device is ready to transfer data or not. (1) – ready (0) – not ready, µp waits, used for interfacing low speed peripherals.

- **DMA signals**

  HOLD – The signal indicates that another device requests the use of data and address bus. When µp recieves the HOLD signal, it relinquishes the use of data and address bus as soon as current machine cycle is completed and the requesting device gets access to the bus. After removal of HOLD, the µp regains the bus. During HOLD, internal processing by µp continues.

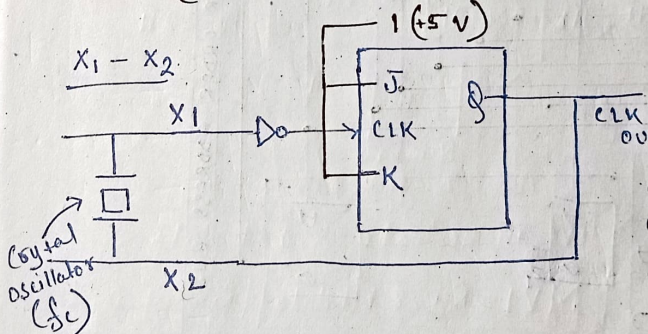
HLDA – It is an acknowledgement sent by the µp after receiving HOLD request. On removal of HOLD, HLDA goes low.

~~Serial I/O~~

6. Serial I/O signals:

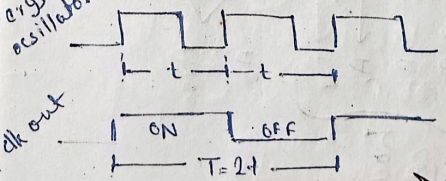
SOD (serial output data) – send single bit data

SID (serial input data) – receive single bit data



S1	SO	OP
0	0	Halt
0	1	Write
1	0	Read
1	1	Fetch

8085 µp needs 3MHz clock, it needs crystal oscillator of frequency – 6MHz.

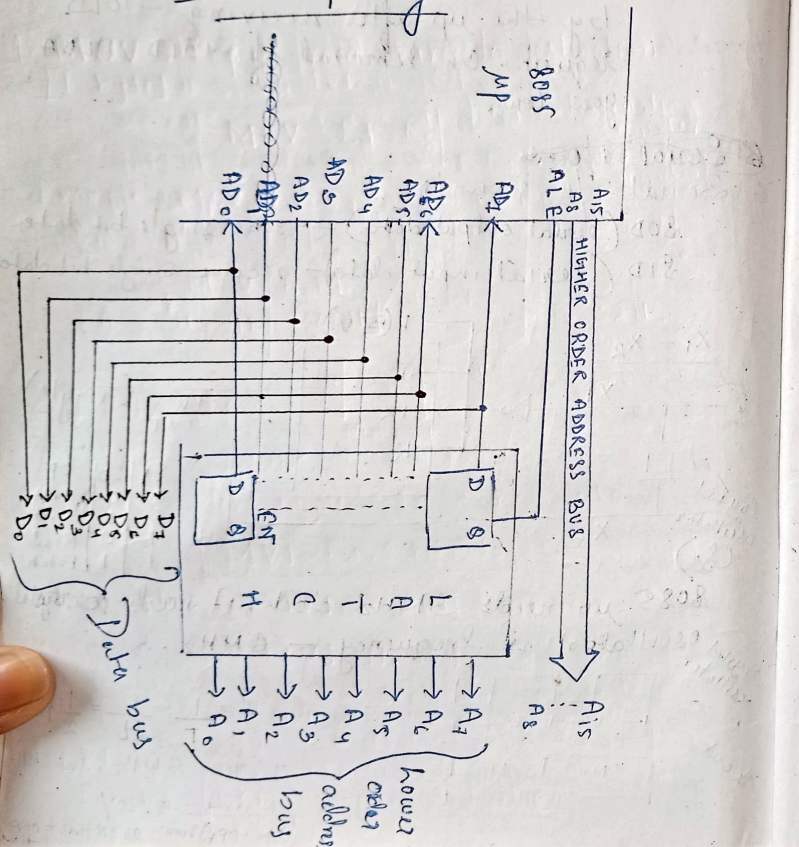


$f = \frac{1}{T} = \frac{1}{2t} = \frac{1}{2} fc$

If f = 3MHz, fc = 6MHz

Dc = $\frac{\text{ON time}}{(\text{ON} + \text{OFF}) \text{ time}}$ → 50% as ON time = OFF time

Demultiplexing circuit



INSTRUCTIONS

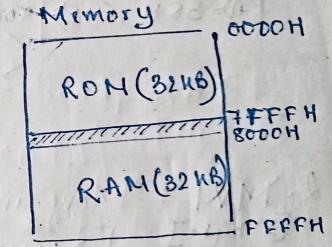
Instruction

- 1 byte — ex. MOV A,B (8 bit)
- 2 byte — ex. MVI A,90 (8 bit, 8 bit)
- 3 byte — ex. LDA 9000H (8 bit, 16 bit)

- → Data transfer instructions
- → Arithematic and logical instructions
- → Branching instructions
- → Machine control instructions.

User language → Mnemonics → (Assembler) → Machine code → µP

Memory



- Programmable register – A, B, C, D, E, H, L
- Memory register — M [when we use M, µP understands the address of register is stored in HL]

DATA TRANSFER INSTRUCTIONS

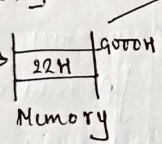
1. MOV : Copy from source to destination
[1 byte]

MOV R_d, R_s

[R_d – destination register
R_s – source register]

Example : i) MOV A, B
[(A) = 35H (B) = 90H] → after → [(A) = 90H, (B) = 90H]

ii) MOV A, M
[(A) = 35H, (HL) = 9000H] → (A) = 22H.



2. MVI : Move immediate 8-bit data.
[2 byte]

MVI R_d, 8-bit data

[R_d – dest. reg.]

Example : i) MVI A, 90H → (A) = 90H.

ii) MVI M, 90H (M → address in HL)
∴ First we have to initialize HL

MVI H, 90H
MVI L, 00H
(HL) = 9000H

3. LXI : Load immediate 16 bit value
[3 byte]

LXI Reg-pair, 16-bit address

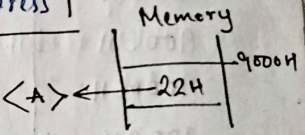
Example : LXI H, 9000H → (HL) = 9000H

[LXI H/B/D – valid].

4. LDA : Load accumulator direct
[3 byte]

LDA 16-bit address

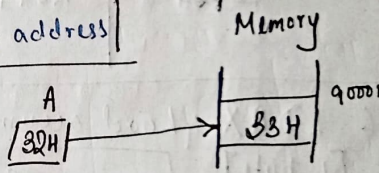
Example LDA 9000H



5. STA : Store accumulator direct.
[3 byte]

STA 16-bit address

Example : STA 9000H



Q. WAP to store 90H to accumulator then transfer its content to C and to memory location 8500H using direct addressing mode. Also transfer the content of C to memory location 8600H using indirect addressing mode.

Ans

MEMORY ADDRESS	MNEMONICS	REMARKS
9000H	MVI A, 90H	<A> ← 90H
9002H	MOV C, A	<C> ← <A>
9003H	STA 8500H	<A> → (8500)H
9006H	LXI H, 8600H	<HL> ← 8600H
9009H	MOV M, C	<C> → (<HL>)
900AH	RST1 / HALT	END

6. LDAX : Load accumulator indirect
[1 byte] LDAX Register-pair

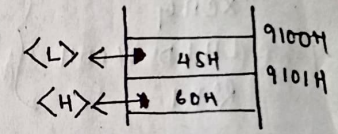
Example : LDAX B <AC> ← (<BC>)
LDAX D <AC> ← (<DE>)
LDAX H X invalid ← MOV A,M

7. STAX : Store Accumulator indirect
[1 byte] STAX reg-pair

Example STAX B <AC> → (<BC>)
STAX D <AC> → (<DE>)
STAX H X invalid ← MOV M,A

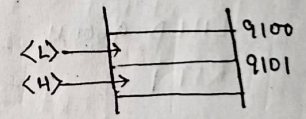
8. LDHLD : Load HL register pair direct.
[3 byte] LDHD 16-bit memory address

Example : LHLD 9100H

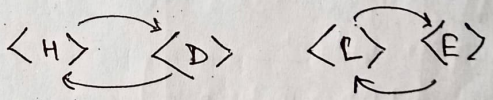


9. SHLD : Store HL register pair direct.
[3 byte] SHLD 16-bit memory address

Example : SHLD 9100H



10. XCHG : Exchange HL content with DE
[1 byte]
Example : XCHG [<H> = 30H, <L> = 20H, <D> = 00H, <E> = 10H]



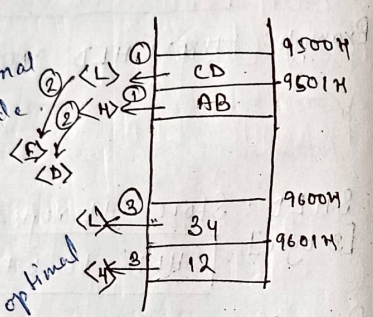
After - [<H> = 00H, <L> = 10H, <D> = 30H, <E> = 20H]

Q. WAP to collect 2 16-bit values from memory location available from 9500H and 9600H.

Ans.

```
LHLD 9500H
XCHG
LHLD 9600H
RST 1
```

optimal code




```
LHLD 9500H
MOV D,H
MOV E,L
LHLD 9600H
RST 1
```

not optimal

ARITHMETIC INSTRUCTIONS

1. ADD : Add register / memory content to accumulator
   [1 byte] — ADD register/M
   ex: ADD B — <A> + <B> → <A>
   ADD M — <A> + (<HL>) → <A>

2. ADI : Add immediate 8-bit data to accumulator
   [2 byte] — ADI 8-bit data
   ex ADI 90H — <A> + 90H → <A>

3. ADC : Add register / memory content to accumulator with carry.
   [1 byte] — ADC register/M
   ex. ADC B — <A> + <B> + <CY> → <A>

4. ACI : Add immediate 8-bit data to accumulator with carry.
   [2-byte] — ACI 8-bit data
   ex ACI 90H — <A> + 90H + <CY> → <A>

5. SUB : Subtract register / memory content from accumulator
   [1] — SUB register/M
   ex. SUB B — <A> − <B> → <A>

6. SUI : Subract immediate 8-bit data from accumulator
[2] | SUI 8-bit data |

ex: SUI 90H <A> − 90H ⟶ <A>

7. SBB : Subtract register/memory content from
[1] accumulator with carry.

| SBB register/M |

ex: SBB D <A> − <D> − <CY> ⟶ <A>

8. SBI : Subtract 8-bit data value from accumulator
[2] with carry.

| SBI 8-bit data |

ex SBI 85H <A> − 85H − <CY> ⟶ <A>

9. DAD [Double addition] : Add HL content to
[1] register pair content

| DAD register-pair |

ex DAD B <HL> + <BC> ⟶ <HL>

10. INR : Increment register/memory content by 1
[1] | INR register/M | [cannot modify <CY> flag]

ex. INR B <B> + 01 ⟶ <B>

11. INX : Increment register-pair content by 1.
[1] | INX register-pair |

ex: INX H <HL> + 01 ⟶ <HL>

12. DCR : Decrement register/memory content by 1
[1] | DCR register/M | [cannot modify <CY> flag]

ex. DCR M (<HL>) − 01 ⟶ (<HL>)

13. DCX : Decrement register-pair content by 1.
[1] | DCX register-pair |

ex: DCX B <BC> − 01 ⟶ <BC>

Q. WAP to add a number which is stored in Memory location 8200 with another number which is stored in memory location 8300. Store the result from 8500.

Memory address	Mnemonics	Remarks
9000H	LDA 8200H	$\langle A \rangle \leftarrow (8200)_H$
9003H	MOV B, A	$\langle B \rangle \leftarrow \langle A \rangle$
9004H	LDA 8300H	$\langle A \rangle \leftarrow (8300)_H$
9007H	ADD B	$\langle A \rangle \leftarrow \langle A \rangle + \langle B \rangle$
9008H	STA 8500H	$(8500)_H \leftarrow \langle A \rangle$
900BH	MVI A, 00H	$\langle A \rangle \leftarrow 00$
900DH	ADC A	$\langle A \rangle \leftarrow \langle A \rangle + \langle A \rangle + \langle CY \rangle$
9014H	STA 8501H	$(8501)_H \leftarrow \langle A \rangle$
9017H	RST 1/HALT	END

Q. WAP to subtract a no which is stored in 8200 from a no which is stored in 8300. Store the difference in 8500 and borrow 8501

* USING SUB:

9000H	LDA 8200H	$\langle A \rangle \leftarrow (8200)_H$
9003H	MOV B, A	$\langle B \rangle \leftarrow \langle A \rangle$
9004H	LDA 8300H	$\langle A \rangle \leftarrow (8300)_H$
9007H	SUB B	$\langle A \rangle \leftarrow \langle A \rangle - \langle B \rangle$
9008H	STA 8500H	$(8500)_H \leftarrow \langle A \rangle$
900BH	MVI 00H	$\langle A \rangle \leftarrow 00$
900DH	ADC A	$\langle A \rangle \leftarrow \langle A \rangle + \langle A \rangle + \langle CY \rangle$
9014H	STA 8501H	$(8501)_H \leftarrow \langle A \rangle$
9017H	RST 1/HALT	END

* Using 2's complement

9000H	LDA 8200H	$\langle A \rangle \leftarrow (8200)_H$
9003H	CMA	$\langle A \rangle \leftarrow \overline{\langle A \rangle}$
9004H	MOV B, A	$\langle B \rangle \leftarrow \langle A \rangle$
9005H	LDA 8300H	$\langle A \rangle \leftarrow (8300)_H$
9008H	ADD B	$\langle A \rangle \leftarrow \langle A \rangle + \langle B \rangle$
9009H	STA 8500H	$(8500)_H \leftarrow \langle A \rangle$
900CH	MVI CNC 9012	$\langle A \rangle \leftarrow 00$
900EH	ADC A	$\langle A \rangle \leftarrow \langle A \rangle + \langle A \rangle + \langle CY \rangle$
9011H	STA 8501H	$(8501)_H \leftarrow \langle A \rangle$
9014H	RST 1/HA	END

Logical Instructions

1. DAA : Decimal adjust accumulator.
   Convert Binary to BCD value.

2. CMP : Compare register/memory content with
   [1] accumulator
   CMP register/M

   ex CMP B <A> compared with <B>

3. CMI : Compare immediate 8-bit data with accumulator
   [2] CMI 8 bit data

   ex CMI 90H <A> compared with 90H.

For CMP and CMI → <A> - value

	CY	Z
<A> > value,	0	0
<A> = value,	0	1
<A> < value,	1	0

4. ANA : Logical AND register/memory content
   [1] with accumulator.
   ANA register/M

5. ANI : Logical AND 8-bit data with accumulator
   [2] ANI 8-bit data

6. XRA : Logical XOR register/memory content with
   [1] accumulator.
   XRA register/M

7. XRI : logical XOR 8-bit data with accumulator.
   [2] XRI 8-bit data

8. ORA : Logical OR register/memory content with
   [1] accumulator
   ORA register/M

9. ORI : Logical OR 8-bit data with accumulator
   [2] ORI 8-bito data

10. CMA : Complement accumulator content
    [1]

11. CMC : Complement carry flag.
    [1]

BRANCHING INSTRUCTIONS.

JUMP – Instruction [3-byte]

Conditional	Unconditional
• JMP	• JNC • JNZ • JM • JPE • JC • JZ • JP • JPO

JMP : Jump immediate to 16-bit address

JMP 16-bit address

ex. JMP 9000H

JNC : Jump if carry flag is 0

JNC 16-bit address

ex.

JC : Jump if carry flag is 1

ex

JNZ : Jump if zero flag is 0

ex

JZ : Jump if zero flag is 1.

JM : Jump if sign flag is 1

ex.

JP : Jump if sign flag is 0

ex.

JPE : Jump if parity flag is 1

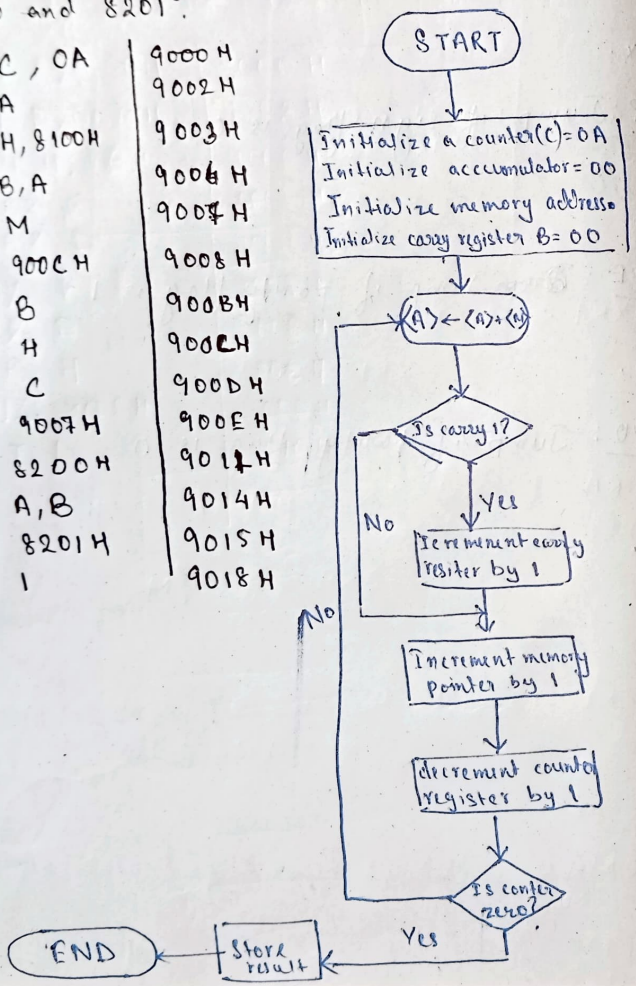
ex.

JPO : Jump if parity flag is 0

ex.

Q.1. WAP to add 10 nos stored in successive memory location starting at 8100. Store the result in 8200 and 8201.

MVI C, 0A	9000 H
SUB A	9002 H
LXI H, 8100H	9003 H
MOV B, A	9006 H
ADD M	9007 H
JNZ 900CH	9008 H
INC B	900BH
INX H	900CH
DCR C	900DH
JNZ 9007H	900EH
STA 8200H	9011H
MOV A, B	9014H
STA 8201H	9015H
RST 1	9018H



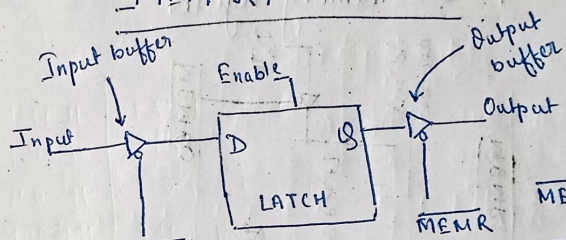
Q.2 8 8-bit numbers are stored in consecutive memory location starting from 8500. Write a program to add them and store result in 8600.

9000 H	MVI C, 08
9002 H	SUB A
9003 H	LXI H, 8500
9006 H	MOV B, A
9007 H	ADD M
9008 H	JNC 900CH
900B H	INC B
900C H	INX H
900D H	DCR C
900E H	JNZ 9007 H
9011 H	STA 8600H
9014 H	MOV A, B
9015 H	STA 8601H
9018 H	RST 1

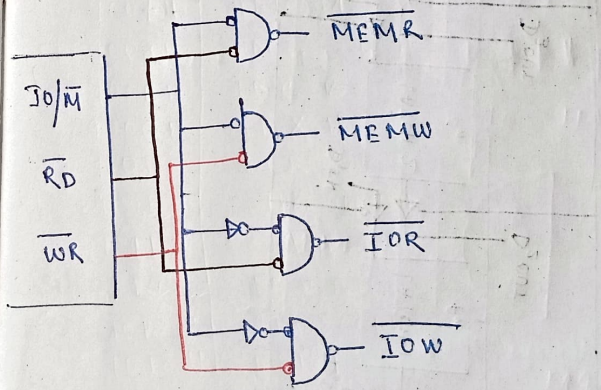
8.8. 6 8-bit data stored in memory location C050H. Transfer the entire block of data to new memory location starting from C070.

LXI B, C050H	9000 H
LXI D, C070H	9003 H
MVI H, 06 H	9006 H
LDAX B	9008 H
STAX D	9009 H
INX B	900A H
INX D	900B H
DCR H	900C H
JNZ 9008 H	900D H
RST 1	9010 H

MEMORY INTERFACE

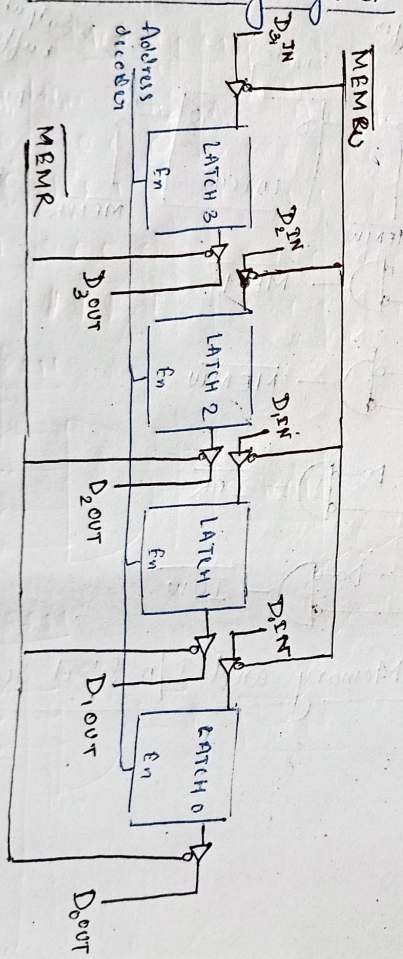


$\overline{MEMW}$ - Memory write
$\overline{MEMR}$ - Memory read
$\overline{IOR}$ - I/O Read
$\overline{IOW}$ - I/O write

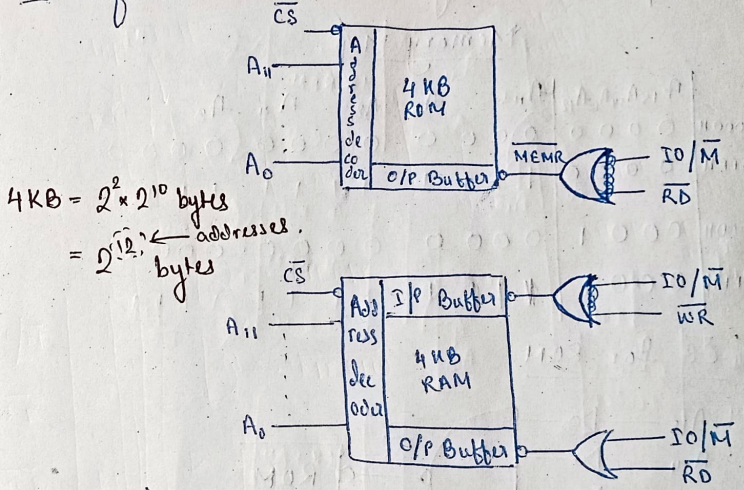


Memory and I/O Select Signal

4 - bit memory register



Interface 4KB RAM and 4KB ROM with 8085 µp



4KB = $2^2 \times 2^{10}$ bytes

= 2^{12} ← addresses.
bytes

Select ($\overline{CS}$) is obtained through two ways.

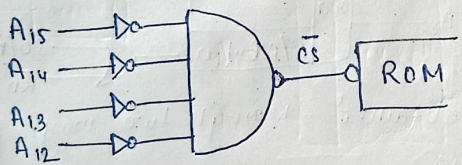
- Gate addressing
- Decoder addressing.

→ Gate Addressing.

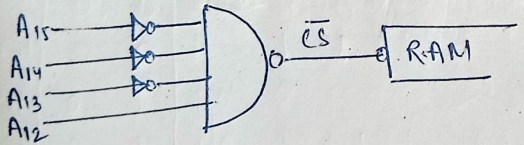
Memory Map.

	$A_{15}A_{14}A_{13}A_{12}$	A_{11} --- A_8	A_8 --- A_4	A_3 --- A_0
0000H	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0
0FFFH	0 0 0 0	1 1 1 1	1 1 1 1	1 1 1 1
1000H	0 0 0 1	0 0 0 0	0 0 0 0	0 0 0 0
1FFFH	0 0 0 1	1 1 1 1	1 1 1 1	1 1 1 1

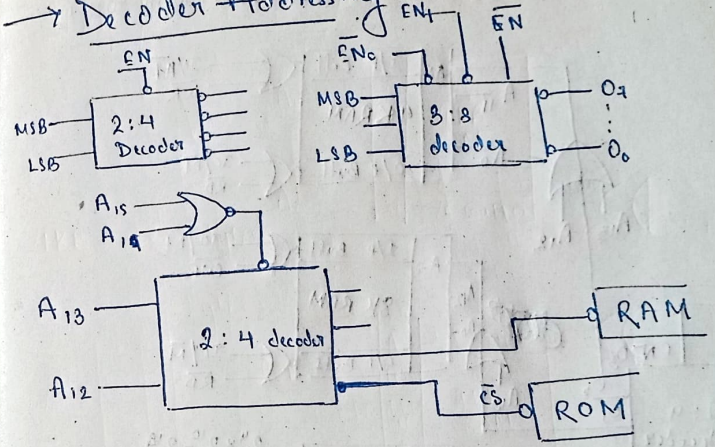
$\overline{CS}$ for ROM



$\overline{CS}$ for RAM



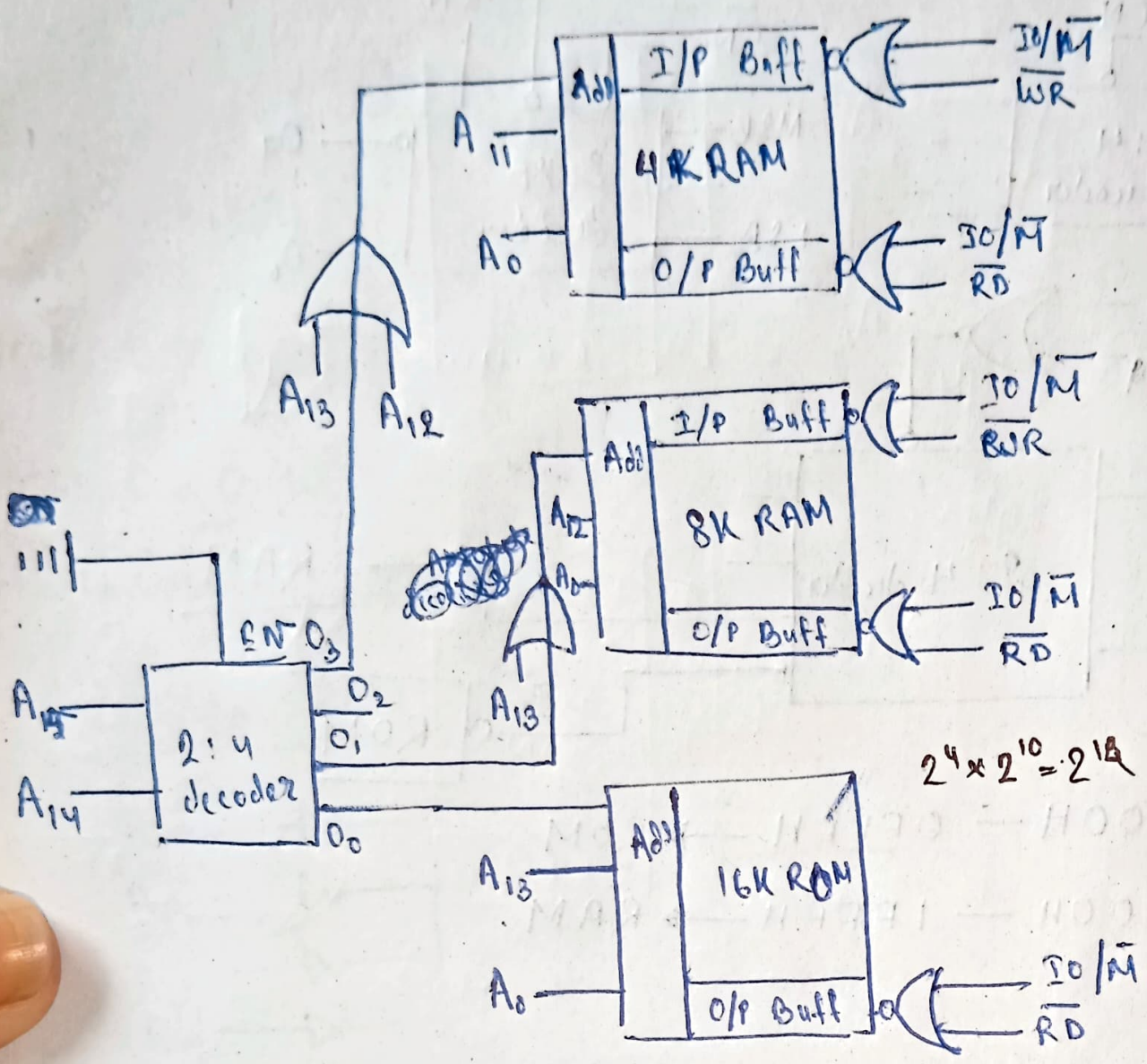
→ Decoder Addressing



0000H — 0FFFH → ROM

1000H — 1FFFH → RAM

Interface 4K RAM, 8K RAM and 16K EPROM with 8085 µP



Memory Map

	A_{15}	A_{14}	A_{13}	A_{12}	$A_{11}-A_8$	A_7-A_4	A_3-A_0
16K ROM							
0000H	0	0	0	0	0000	0000	0000
3FFFH	0	0	1	1	1111	1111	1111
4000H	0	1	0	0	0000	0000	0000
5FFFH	0	1	0	1	1111	1111	1111
C000H	1	1	0	0	0000	0000	0000
CFFFH	1	1	0	0	1111	1111	1111